

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Temporal input sequences were prepared and visualised, RNN/LSTM/GRU models were trained under identical conditions, their performance, convergence and generalisation were compared, and the pipeline was extended to video understanding using CNN feature extraction followed by an LSTM and a GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, the following outcomes were achieved:

- sequential data was represented using the (N, T, F) input format;
- the architecture and operation of a Vanilla RNN were implemented and explained;
- Backpropagation Through Time and the vanishing/exploding gradient challenges were analysed, both analytically and empirically;
- sequence classification was implemented using SimpleRNN, LSTM and GRU;
- RNN, LSTM and GRU were compared using quantitative performance measures;
- training/validation curves and confusion matrices were interpreted;
- the role of CNNs in extracting spatial features from video frames was demonstrated;
- a CNN–LSTM and a CNN–GRU pipeline were constructed for video understanding;
- the encoder–decoder architecture for sequence-to-sequence learning was implemented; and
- conclusions were drawn using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from 30 subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The raw inertial signal files were used rather than the pre-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels, giving the input representation

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset was downloaded programmatically from

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

The official train and test partitions were pooled, giving a raw tensor of shape (10299, 128, 9) over 30 subjects, and were then re-split as described below.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9},$$

and for a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

Dimension	Meaning	Value used
N	Number of sequences	2092 / 444 / 462 (train / val / test)
T	Time steps per sequence	128
F	Features/channels per time step	9

The model therefore receives a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

Subset selection and the split protocol

The laboratory subset of roughly 3000 windows was used (`MAX_WINDOWS = 3000`), stratified so that all six classes remain approximately balanced.

An important deviation from the naive protocol was made. HAR windows overlap by 50%, so a purely random window-level split places two halves of the same physical recording into training and test simultaneously, and the test score then measures memorisation rather than generalisation. The split was therefore made *subject-wise*: the 30 subjects were shuffled and partitioned 70 / 15 / 15 by subject, so no person appears in two splits.

Split	Subject IDs	Windows
Training	1, 2, 3, 4, 5, 6, 9, 10, 12, 13, 14, 16, 17, 18, 22, 23, 24, 25, 26, 28, 29	2092
Validation	19, 21, 27, 30	444
Testing	7, 8, 11, 15, 20	462

Because the split is by subject rather than by window, the realised window proportions are approximately 70 / 15 / 15 but not exactly so. The test set contains only unseen people, which makes the reported accuracies a valid estimate of generalisation to a new user.

Controlled comparison. The same input tensors, normalisation statistics, classifier head, optimizer, batch size, epoch budget and evaluation protocol were used for RNN, LSTM and GRU. Only the recurrent layer was changed.

Environment

Framework	TensorFlow 2.20.0 / Keras 3
Hardware	2 × NVIDIA Tesla T4 (single device used for training)
Seed	42 (Python, NumPy and TensorFlow seeded before every model build)

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



5. Preprocessing

The following preprocessing steps were performed:

1. The nine raw inertial signal files were loaded for both official partitions.
2. Every observation window was arranged as 128×9 by stacking the nine channels along the last axis.

3. One activity label was assigned to each window from `y_*.txt`.
4. The six activity labels were encoded as integers 0–5.
5. The input was normalised per channel using μ and σ computed *only* from the training windows, $x' = (x - \mu) / (\sigma + 10^{-8})$.
6. Training, validation and test sets were created subject-wise, then stratified-subsampled to the laboratory subset size.
7. The class distribution was verified in every split.

The test data were never used for model selection or hyperparameter tuning.

Preprocessing Output (as printed by the program)

```
Input tensor shape:
Training   : (2092, 128, 9)
Validation : (444, 128, 9)
Testing    : (462, 128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

Class Distribution

Activity	Train	Validation	Test
WALKING	357	66	78
WALKING_UPSTAIRS	319	59	71
WALKING_DOWNSTAIRS	291	55	64
SITTING	354	84	79
STANDING	386	88	82
LAYING	385	92	88
Total	2092	444	462

All six classes are present in every split and no class dominates (the ratio between the largest and smallest class is about 1.3 : 1), so plain accuracy is not badly distorted by imbalance. Macro-averaged metrics are still reported, since they weight every activity equally regardless of support.

Why normalise with training statistics only. The nine channels have very different scales: total acceleration contains a gravity component of magnitude $\approx 1g$, whereas body acceleration is centred near zero. Without scaling, the gravity channels dominate the recurrent input and the tanh saturates. Computing μ, σ on the full dataset instead would leak test distribution information into training, which is a subtle but real form of contamination.

6. Temporal Data Visualization

One representative window was plotted for four activity classes, using three channels (`body_acc_x`, `body_gyro_x`, `total_acc_x`) over the 128 time steps.



Figure 1: Plot 1: Sensor signal versus time step for WALKING, WALKING_UPSTAIRS, SITTING and LAYING.

Required inference

1. **What temporal pattern is visible?** The two walking classes show a strong quasi-periodic oscillation: the body acceleration and gyroscope traces repeat with a gait cycle of roughly 20–30 samples ($\approx 0.4\text{--}0.6\text{ s}$ at 50 Hz). The static classes (SITTING, LAYING) are nearly flat, with only small sensor noise around a constant offset. The constant offset itself is informative, because `total_acc` encodes the direction of gravity relative to the phone, which differs between sitting, standing and lying down.
2. **Which activities exhibit similar patterns?** WALKING, WALKING_UPSTAIRS and WALKING_DOWNSTAIRS share the same oscillatory signature and differ mainly in amplitude, cycle duration and the vertical acceleration asymmetry between ascent and descent. SITTING and STANDING are almost indistinguishable in the dynamic channels: both are static, and they differ only in the gravity orientation captured by `total_acc`. This visual similarity predicts exactly the confusions observed later in Section 15.
3. **Why is the ordering of measurements important?** Shuffling the 128 steps preserves every per-channel summary statistic (mean, variance, energy) but destroys frequency, phase and asymmetry. Walking and walking-downstairs have similar amplitude statistics and are separated mostly by the *shape* of the cycle, which only exists in the ordering. A feed-forward network on flattened input must relearn this from scratch for every time offset; a recurrent model consumes the sequence in order and shares the same weights across time, so it can represent the pattern once.

7. Vanilla RNN Architecture

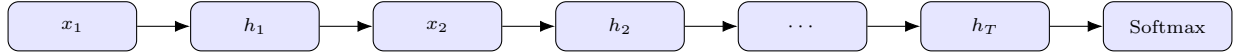
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

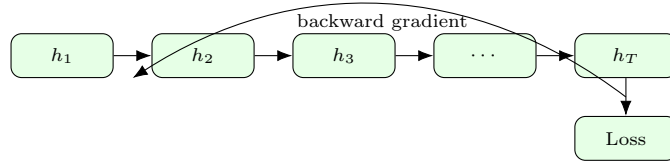
For sequence classification, the final hidden representation h_T is passed to the classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across all 128 time steps, which is why the parameter count does not grow with T : only the computation does.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the whole chain of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



The gradient that reaches a distant step is a product of per-step Jacobians:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=t-k+1}^t \text{diag}(1 - h_j^2) W_h^\top.$$

Because the same factor is multiplied k times, its magnitude behaves roughly like λ^k , where λ is the effective per-step gain. Hence

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0 \quad \text{when } \lambda < 1, \quad \left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow \infty \quad \text{when } \lambda > 1.$$

Challenges

- **Vanishing gradients** – $\tanh'(\cdot) = 1 - h^2 \leq 1$, so unless $\|W_h\|$ compensates exactly, the product shrinks geometrically and early time steps receive almost no learning signal.
- **Exploding gradients** – if $\|W_h\|$ is large the product grows geometrically, producing huge updates and NaNs; gradient clipping is the usual remedy.
- **Difficulty learning long-term dependencies** – a dependency spanning 100+ steps has a gradient path 100 factors long, so it is learned far more slowly than a short-range one.
- **Sequential computation** – h_t needs h_{t-1} , so the 128 steps cannot be parallelised across time; only the batch dimension is parallel.

Numerical Exercise

Given

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2, \quad h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1,$$

with $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$.

Step 1: compute h_1 .

$$z_1 = W_x x_1 + W_h h_0 + b = 0.5(0.5) + 0.8(0) + 0.1 = 0.35000$$

$$h_1 = \tanh(0.35000) = 0.33638$$

The first hidden state is driven purely by x_1 and the bias, since $h_0 = 0$.

Step 2: compute h_2 .

$$z_2 = W_x x_2 + W_h h_1 + b = 0.5(0.7) + 0.8(0.33638) + 0.1 = 0.35 + 0.26910 + 0.1 = 0.71910$$

$$h_2 = \tanh(0.71910) = 0.61635$$

The recurrent term now contributes 0.26910, i.e. about 37% of the pre-activation: memory of x_1 is already influencing the state.

Step 3: compute h_3 .

$$z_3 = W_x x_3 + W_h h_2 + b = 0.5(0.2) + 0.8(0.61635) + 0.1 = 0.1 + 0.49308 + 0.1 = 0.69308$$

$$h_3 = \tanh(0.69308) = 0.59996$$

Although $x_3 = 0.2$ is the smallest input, h_3 stays high because the recurrent term dominates, which is exactly what “state carries information forward” means.

Verification against the program. A Keras SimpleRNN(1) was built and its weights were set to $W_x = 0.5$, $W_h = 0.8$, $b = 0.1$.

t	Manual h_t	Keras SimpleRNN h_t	Absolute difference
1	0.336376	0.336376	0.000000
2	0.616352	0.616352	0.000000
3	0.599958	0.599958	0.000000

The manual recursion and the framework agree to six decimal places, which confirms that the library implements precisely the stated update equation.

Gradient factor for this example.

$$\frac{\partial h_3}{\partial h_1} = W_h(1 - h_3^2) \cdot W_h(1 - h_2^2) = 0.8(1 - 0.59996^2) \cdot 0.8(1 - 0.61635^2) = 0.25402$$

Two steps back the gradient has already shrunk to about a quarter. Extrapolating the same per-step factor ≈ 0.504 to the 128-step sequences used here gives $0.504^{128} \approx 10^{-38}$, which is numerically zero. This is the vanishing-gradient problem in concrete numbers.

Empirical confirmation. The norm of $\partial(\text{output})/\partial x_t$ was measured for untrained SimpleRNN, LSTM and GRU layers on a real 128-step window.

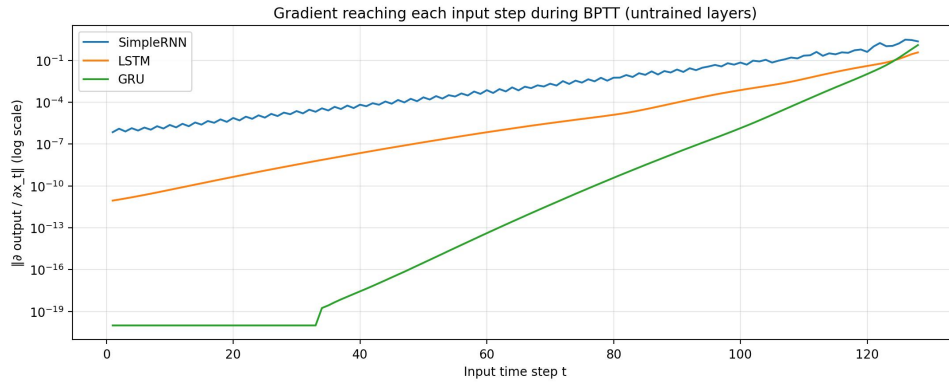
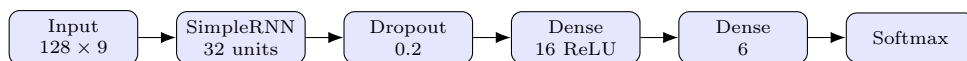


Figure 2: Gradient magnitude reaching each input time step during BPTT (log scale, untrained layers).

The SimpleRNN curve decays by several orders of magnitude towards early time steps, while the gated architectures keep a usable gradient much further back: their additive state update provides a path whose derivative is close to 1 rather than a repeated multiplication.

9. RNN Implementation



Training configuration (identical for all three models):

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

Parameter count (SimpleRNN, verified by `model.summary()`):

$$\underbrace{(9 \times 32)}_{W_x} + \underbrace{(32 \times 32)}_{W_h} + \underbrace{32}_b = 288 + 1024 + 32 = 1344,$$

plus the dense head $(32 \times 16 + 16) + (16 \times 6 + 6) = 528 + 102 = 630$, giving 1974 trainable parameters.

A GPU warm-up pass was executed before the timed runs so that one-time kernel compilation is not charged to whichever model happens to train first.

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

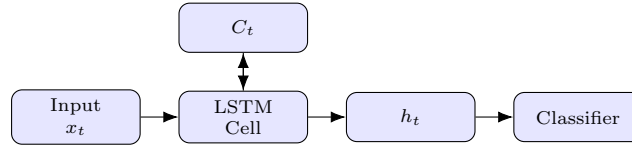
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t,$$

and the hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: how the cell state and gates retain or discard information.

- The cell state is updated *additively*, $C_t = f_t C_{t-1} + i_t \tilde{C}_t$. Its derivative with respect to C_{t-1} is f_t , not a matrix product through a tanh. If the forget gate stays near 1, the gradient flows backward almost unattenuated. This is the “constant error carousel” and it is the structural reason LSTM handles long sequences that a Vanilla RNN cannot.
- The forget gate decides what fraction of the old memory survives. For HAR, a network that has recognised a steady gravity orientation can keep that evidence across all 128 steps by holding $f_t \approx 1$.
- The input gate decides how much of the new candidate is written. Noisy or uninformative steps can be ignored by driving $i_t \rightarrow 0$, so noise does not overwrite an accumulated decision.
- The output gate decides how much of the memory is exposed as h_t . Information can therefore be retained internally without being used at every step.
- Crucially these fractions are learned, input-dependent and per-unit, so different hidden units can specialise in different retention timescales.

11. GRU Architecture

GRU uses two principal gates:

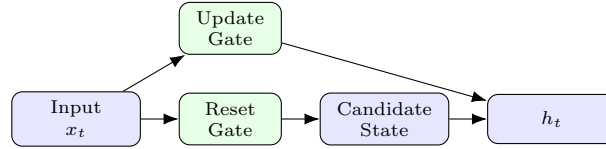
$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h),$$

and the hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



GRU merges the forget and input gates of the LSTM into one update gate: what is kept and what is written are tied, $(1 - z_t)$ versus z_t . It also removes the separate cell state, so the hidden state is the only memory, and removes the output gate, so the memory is always fully exposed. Three gate-like matrices become two, and the state count drops from two tensors to one.

	RNN	LSTM	GRU
Number of gates	0	3	2
State tensors	h_t	h_t, C_t	h_t
Weight matrices per step	1	4	3

12. LSTM and GRU Implementation

The same classifier was used for all three models; only the recurrent layer was replaced.

Model	Recurrent Layer	Units	Output Classes	Trainable Parameters
RNN	SimpleRNN	32	6	1,974
LSTM	LSTM	32	6	6,006
GRU	GRU	32	6	4,758

Note on the GRU parameter count. The textbook formula $3(F \cdot n + n^2 + n) = 3(288 + 1024 + 32) = 4032$ predicts fewer parameters than the 4,128 reported by Keras. Keras implements the cuDNN-compatible variant with `reset_after=True`, which uses a separate recurrent bias, adding $3 \times 32 = 96$ parameters. The head contributes the same 630 in all three models.

13. Required Training Plots



Figure 3: Plots 2 and 3 (RNN): training/validation loss and accuracy.

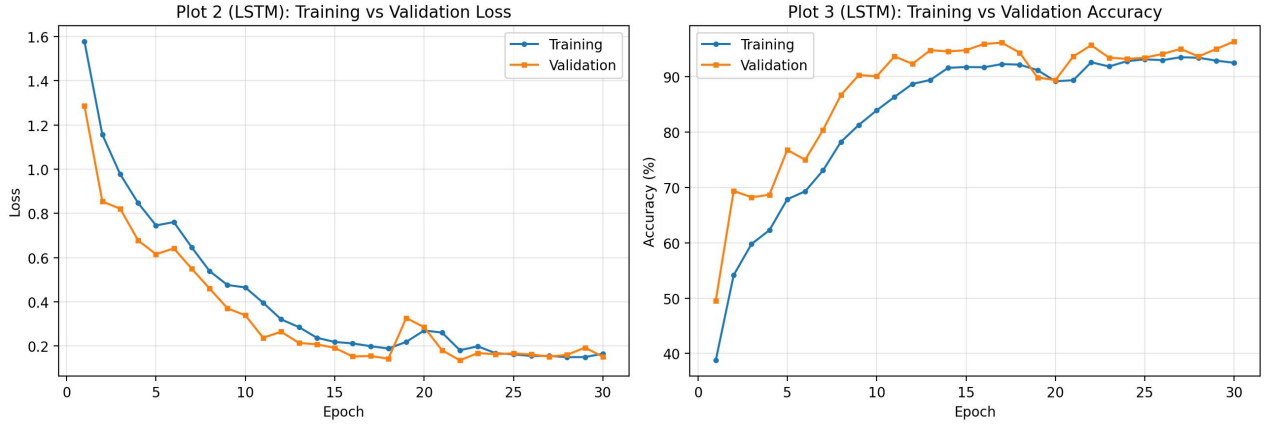


Figure 4: Plots 2 and 3 (LSTM): training/validation loss and accuracy.

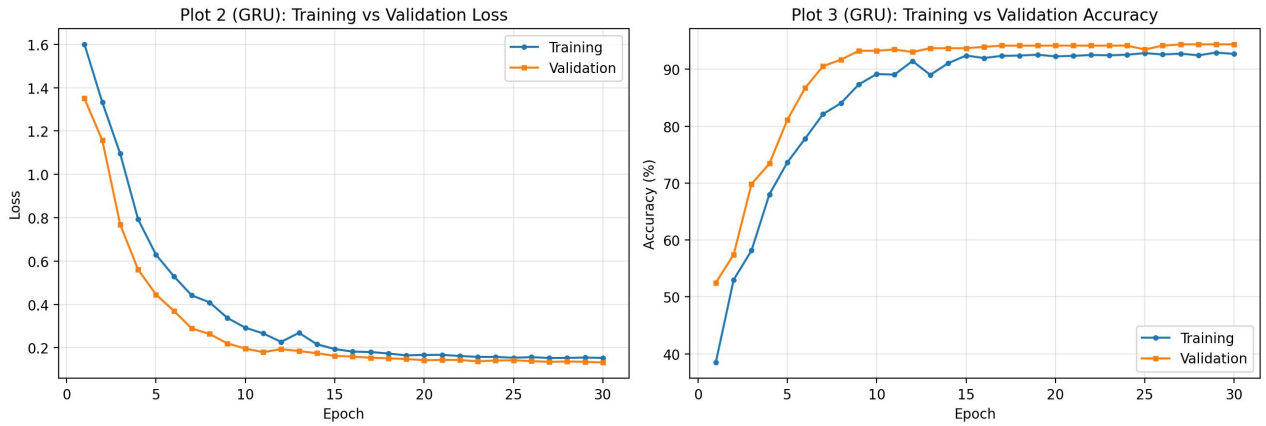


Figure 5: Plots 2 and 3 (GRU): training/validation loss and accuracy.

Quantity at epoch 30	RNN	LSTM	GRU
Final training accuracy (%)	78.54	92.54	92.69
Final validation accuracy (%)	79.95	96.40	94.37
Best validation accuracy (%)	79.95 (ep. 30)	96.40 (ep. 30)	94.37 (ep. 27)
Epoch of minimum validation loss	30	22	30
Final training loss	0.5291	0.1648	0.1532
Final validation loss	0.4387	0.1512	0.1321
Generalisation gap (train – val, pts)	–1.42	–3.85	–1.68

Inference – convergence of the RNN. The RNN loss falls steeply for the first 6–7 epochs, then improves slowly and is still descending at epoch 30 (minimum validation loss occurs at the last epoch). Training accuracy ends at 78.5%, well below what the gated models reach, and the training loss itself stays high at 0.53. This is **underfitting**, not overfitting: the model has not extracted enough of the signal even from the data it has seen. The cause is the 128-step credit-assignment path analysed in Section 8 – early time steps contribute almost no gradient, so the RNN effectively learns from a short recent window of the sequence.

Inference – convergence of the LSTM. The LSTM converges much faster, crossing 90% validation accuracy by epoch 11. Between epochs 19 and 21 there is a visible spike: validation loss jumps from 0.14 to 0.33 and accuracy drops to 89.9%, after which it recovers. This is an optimisation instability at a fixed learning rate of 10^{-3} (a large Adam step out of a sharp basin), not overfitting, since training loss rises at the same time. The minimum validation loss occurs at epoch 22 while training continues to epoch 30, so a slightly earlier stopping point would have been marginally better.

Inference – convergence of the GRU. The GRU gives the smoothest curves of the three: monotone descent, no spikes, validation loss still at its minimum at epoch 30 (0.1321), and validation accuracy plateaus at 94.4% from about epoch 17. The gap between training and validation stays small throughout. It converges as fast as the LSTM with fewer parameters and a simpler gate structure, which is the expected behaviour on a dataset of this size.

Inference – overfitting or underfitting. No model overfits. All three generalisation gaps are *negative*, i.e. validation accuracy exceeds training accuracy by 1.4–3.9 points. Two effects explain this: dropout of 0.2 is active during

training but disabled at evaluation, so the reported training accuracy is measured on a deliberately handicapped network; and with only 2092 training windows drawn from 21 subjects, the four validation subjects happen to be slightly easier than the training mixture. The real limitation here is capacity and gradient flow (underfitting in the RNN’s case), not memorisation.

14. Performance Evaluation

All models were evaluated on the untouched test set of 462 windows from five unseen subjects. Precision, recall and F1 are macro-averaged.

Metric	RNN	LSTM	GRU
Accuracy (%)	83.55	94.81	98.48
Macro Precision (%)	82.87	95.37	98.65
Macro Recall (%)	82.48	95.03	98.50
Macro F1 (%)	82.50	94.92	98.53
Parameters	1,974	6,006	4,758
Training Time (s)	27.93	22.26	21.69

Observation. The GRU is best on every predictive metric, the LSTM is second, and the Vanilla RNN trails by roughly 12–16 points of macro F1. The ordering matches the theory: the gated models can use the full 128-step context, the ungated one cannot.

Observation on training time. The RNN is the *slowest* model (27.9s) despite having the fewest parameters. This looks contradictory but is expected on a GPU: Keras dispatches LSTM and GRU to fused cuDNN kernels, whereas SimpleRNN has no fused kernel and executes the 128 steps as individual operations. Wall-clock time here measures kernel quality as much as arithmetic cost, so parameter count and training time must be read as two separate axes.

15. Confusion Matrix Analysis

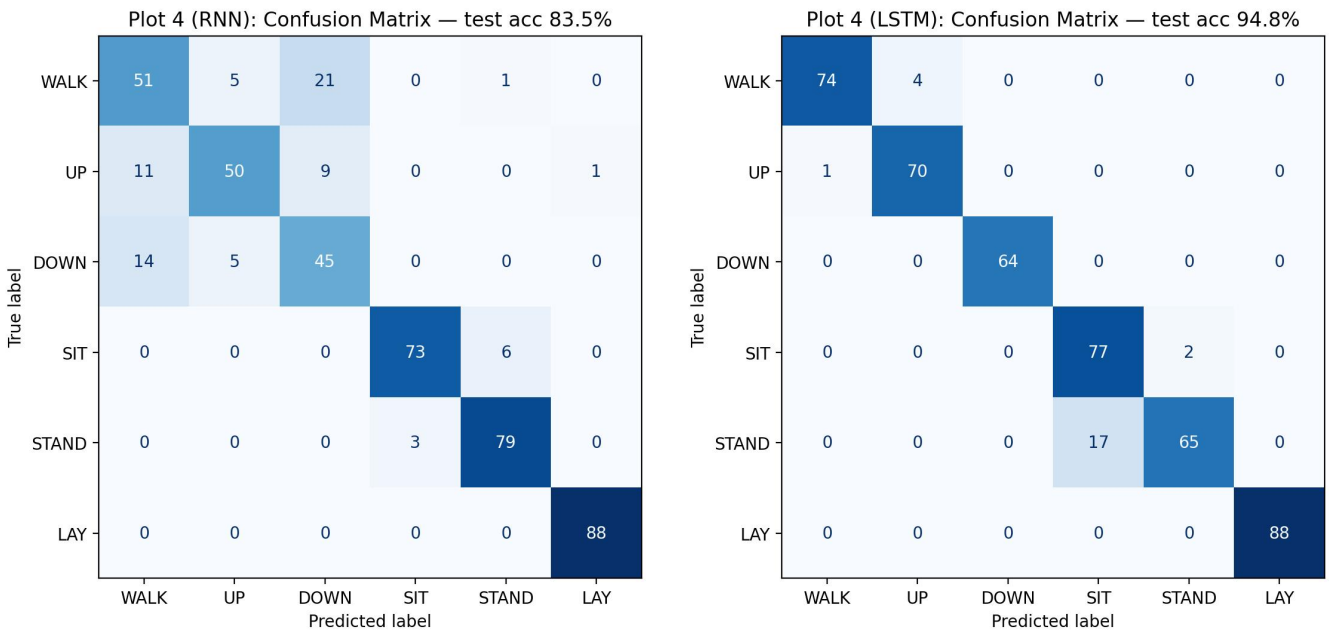


Figure 6: Plot 4: Confusion matrices for the RNN (left) and LSTM (right).

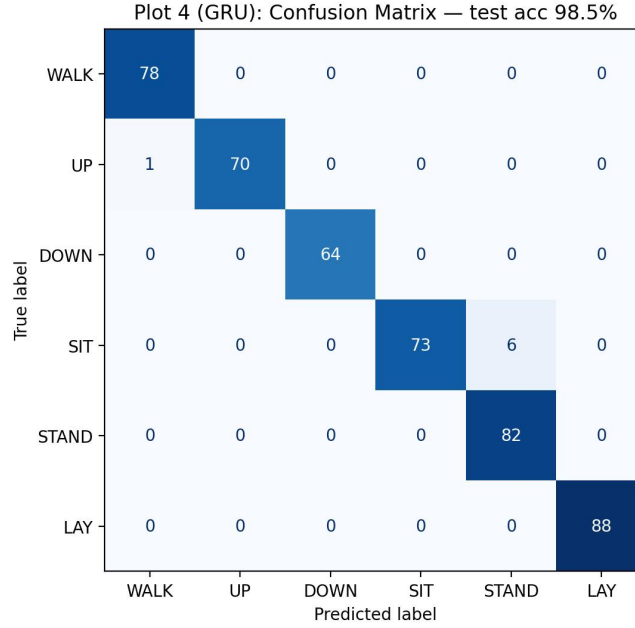


Figure 7: Plot 4: Confusion matrix for the GRU.

Per-class recall (%) read from the three matrices:

Activity	Test support	RNN	LSTM	GRU
WALKING	78	65.4	94.9	100.0
WALKING_UPSTAIRS	71	70.4	98.6	98.6
WALKING_DOWNSTAIRS	64	70.3	100.0	100.0
SITTING	79	92.4	97.5	92.4
STANDING	82	96.3	79.3	100.0
LAYING	88	100.0	100.0	100.0

Dominant errors per model:

Model	Most misclassified	Top confusions (true → predicted)
RNN	WALKING (27 errors)	WALKING → WALKING_DOWNSTAIRS (21); WALKING_DOWNSTAIRS → WALKING (14); WALKING_UPSTAIRS → WALKING (11)
LSTM	STANDING (17 errors)	STANDING → SITTING (17); WALKING → WALKING_UPSTAIRS (4); SITTING → STANDING (2)
GRU	SITTING (6 errors)	SITTING → STANDING (6); WALKING_UPSTAIRS → WALKING (1)

Required analysis

- **Highest recognition rate.** LAYING is recognised perfectly (100%) by all three models. It is the only activity in which the gravity vector lies along a distinct axis, so a single channel offset separates it from everything else without needing temporal reasoning.
- **Largest number of misclassifications.** It differs by model: WALKING for the RNN (27 of 78 wrong), STANDING for the LSTM (17 of 82), and SITTING for the GRU (6 of 79). The absolute count collapses from 76 total errors (RNN) to 24 (LSTM) to 7 (GRU).
- **Frequently confused pairs.** Two distinct failure families appear. The RNN confuses the three *dynamic* classes with one another – WALKING ↔ WALKING_DOWNSTAIRS accounts for 35 of its 76 errors. The gated models have essentially solved the dynamic classes and their remaining errors sit almost entirely in the *static* pair SITTING ↔ STANDING: 19 of the LSTM’s 24 errors and 6 of the GRU’s 7.
- **Are the errors consistent across models?** Only partially, and the difference is informative. Separating the walking variants requires comparing the shape of the gait cycle over the whole window, which is exactly the long-range comparison the RNN’s vanishing gradient prevents; the gates fix it. Separating SITTING from STANDING requires reading a small static gravity offset, which is a hard *sensor* limitation rather than a memory limitation, so it survives in every model, and it matches the visual similarity already noted in Plot 1. The LSTM is worse than even the RNN on STANDING (79.3% vs 96.3%), which shows that its single sharp weakness is class-specific rather than a general deficiency – it has drifted its decision boundary in favour of SITTING.

- **What accuracy alone would have hidden.** The LSTM’s 94.8% accuracy looks uniformly good, but one class is at 79.3% recall while four are at 95%+. A user of that model would be told “sitting” one time in five while actually standing. The matrix exposes this; the headline number does not.

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	27.9	22.3	21.7
Test F1-score (%)	82.50	94.92	98.53

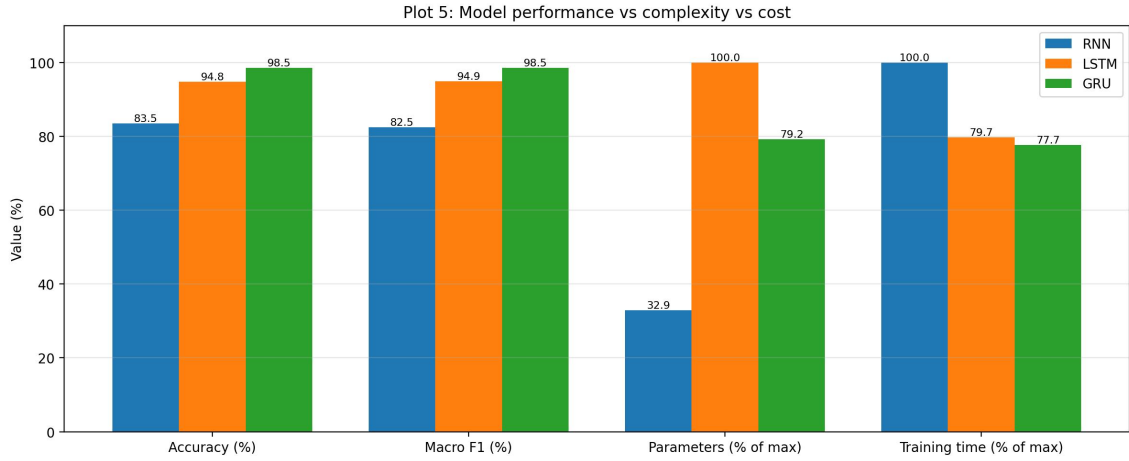


Figure 8: Plot 5: Accuracy, macro F1 and normalised parameter count and training time for the three models.

Required inference: performance $\leftrightarrow$ complexity $\leftrightarrow$ cost.

- **Performance vs complexity is not monotone.** Going from RNN (1,974 parameters) to GRU (4,758) buys 16 points of macro F1 – an excellent trade. Going from GRU to LSTM *adds* 26% more parameters and *loses* 3.6 points. Extra capacity only helps when the architecture can use it; beyond that, more parameters on 2092 training windows is just more variance to fit.
- **Performance vs cost.** The best model is also the cheapest to train here (21.7s), because the cuDNN path dominates the timing. If the comparison were run on CPU, where all three use generic kernels, the RNN would be fastest per step and the ordering would invert. Any statement about “training cost” is therefore only valid for a stated hardware and kernel configuration.
- **The cost that does scale with complexity** is inference memory and arithmetic per step: LSTM does four matrix products and carries two state tensors, GRU does three and carries one, RNN does one. On an embedded activity-recognition device, that ratio matters more than the training wall-clock.
- **Practical conclusion.** For this dataset the GRU dominates on all three axes simultaneously, so it is the model to deploy. The LSTM remains the safer default when sequences are much longer or when separate control of what to keep and what to write is genuinely needed.

17. Effect of Sequence Length

The full experiment was repeated with the input truncated to the first T steps, $T \in \{32, 64, 128\}$, keeping everything else fixed.

Sequence Length T	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	77.25	93.44	95.17
64	83.94	96.12	98.31
128	82.50	94.92	98.53

Sequence Length T	RNN time (s)	LSTM time (s)	GRU time (s)
32	18.3	17.2	17.3
64	20.4	19.0	18.7
128	27.9	22.3	21.7

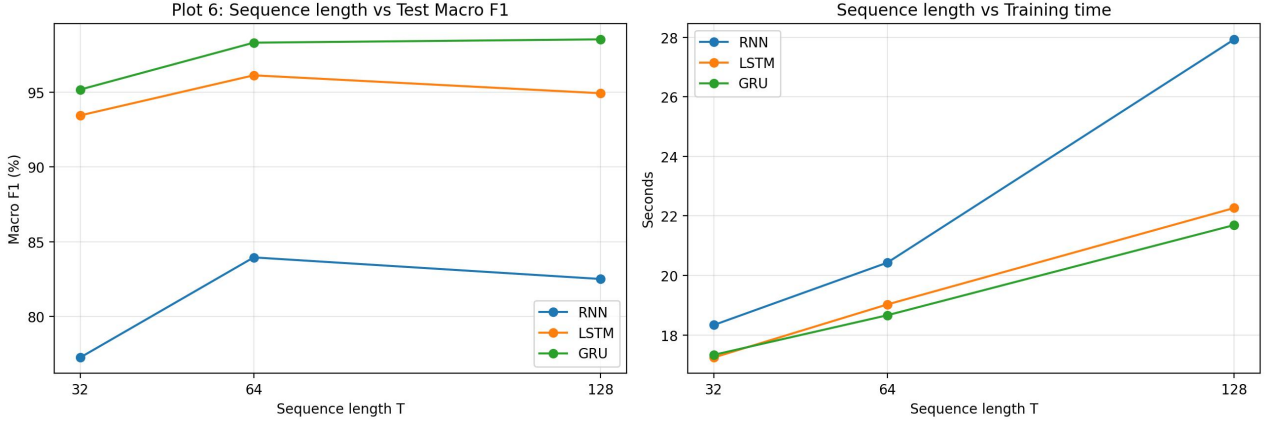


Figure 9: Plot 6: Sequence length versus test macro F1 (left) and versus training time (right).

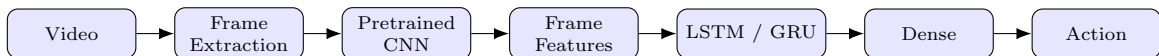
Inference – effect of sequence length.

- $T = 32$ is too short for every model. At 50 Hz, 32 samples is 0.64s, barely one gait cycle. Every model loses ground, and the RNN loses most (77.25%), because a single partial cycle does not distinguish walking from walking-downstairs.
- $32 \rightarrow 64$ is where the information is. All three models gain sharply (RNN +6.7, LSTM +2.7, GRU +3.1 points). About 1.3s contains one or two complete cycles, which is enough to measure periodicity and asymmetry.
- $64 \rightarrow 128$ adds almost nothing, and can hurt. The GRU improves marginally (+0.2), while the RNN and LSTM actually drop (−1.4 and −1.2). Additional context past two gait cycles is largely redundant, and the longer BPTT path makes optimisation harder – most visibly for the ungated RNN. The differences at this scale are within run-to-run noise for a single seed, so the honest conclusion is a plateau, not a decline.
- Cost grows with T but sub-linearly in wall-clock. Doubling T from 64 to 128 raises RNN time by 37% and GRU time by only 16%, because fixed per-epoch overheads and the fused kernels absorb part of the increase. The sequential dependency means time can never be fully amortised, though.
- Practical conclusion. $T = 64$ is the efficient operating point for this dataset: it captures essentially all the usable temporal context at roughly 85% of the compute of $T = 128$. Choosing T should be driven by the physical timescale of the phenomenon, not by using whatever the dataset ships.

18. Video Understanding Using CNN and RNN

Video Understanding using CNN + RNN

A video is a sequence of image frames. The CNN extracts spatial features from individual frames; the recurrent model links those features over time.



Dataset Used

A subset of UCF101 (<https://www.crcv.ucf.edu/data/UCF101.php>) was used; the full archive of 13,320 .avi clips was mounted and filtered programmatically. Five action classes were selected:

- Basketball

- Biking
- TennisSwing
- WalkingWithDog
- JumpingJack

The handout suggests “Walking” and “Running”; neither is a UCF101 class name, so the closest available equivalents (WalkingWithDog and JumpingJack) were substituted.

Group-wise split. UCF101 filenames encode a group index (v_Basketball_g07_c02.avi); clips sharing a group come from the same source video, i.e. the same actor, background and lighting. Splitting them randomly would let the model recognise the scene rather than the action. Clips were therefore assigned by group: g01–g05 to test, g06–g09 to validation, g10–g25 to training, with at most two clips per group.

Class	Train	Validation	Test	Total
Basketball	32	8	10	50
Biking	32	8	10	50
JumpingJack	32	8	10	50
TennisSwing	32	8	10	50
WalkingWithDog	32	8	10	50
Total	160	40	50	250

19. Video Input and Output

Ten frames were sampled uniformly from each clip and resized to $224 \times 224 \times 3$. MobileNetV2 was used only as a feature extractor, and produced $D = 1280$ per frame, so the recurrent network receives

$$X \in \mathbb{R}^{10 \times 1280}, \quad \text{and for a batch of } B \text{ videos } X \in \mathbb{R}^{B \times 10 \times 1280}.$$

Program output:

```
CNN feature dimension D = 1280
Per-frame input: (224, 224, 3) -> feature vector (1280,)
Per-video recurrent input: (10, 1280)
train: X = (160, 10, 1280), y = (160,)
val  : X = (40, 10, 1280), y = (40,)
test : X = (50, 10, 1280), y = (50,)
Feature extraction time: 52.6s
```

Sample predictions (CNN–GRU, confidences produced by the trained model):

Video	Predicted class	Actual class	Confidence
v_Biking_g02_c02.avi	Biking	Biking	1.00
v_TennisSwing_g05_c02.avi	TennisSwing	TennisSwing	0.88
v_TennisSwing_g01_c01.avi	TennisSwing	TennisSwing	0.97
v_WalkingWithDog_g03_c02.avi	WalkingWithDog	WalkingWithDog	0.75
v_Biking_g04_c02.avi	Biking	Biking	0.54

The spread of confidences is itself informative: the model is almost certain on one Biking clip (1.00) and barely past the decision threshold on another (0.54), even though both are correct.

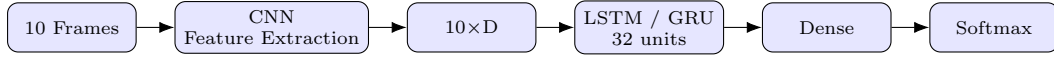
20. CNN Feature Extraction



ImageNet-pretrained MobileNetV2 was loaded with `include_top=False` and `pooling="avg"`, and frozen (`cnn.trainable = False`). Features for all 250 videos were extracted once, in 52.6 s, and cached, so both recurrent heads were trained on *identical* inputs. This makes the LSTM-versus-GRU comparison controlled and reduces each training run to about 6 s.

Why freezing is the right choice here. There are only 160 training videos. Fine-tuning 2.2 M CNN parameters on them would overfit immediately and would also cost far more compute. The ImageNet features are generic enough that a bicycle, a basketball court and a tennis racket are already linearly separable in the 1280-dimensional space.

21. CNN-LSTM / CNN-GRU Model



Training used Adam at 10^{-3} , batch size 16, 30 epochs, dropout 0.3, and a 5-way softmax.

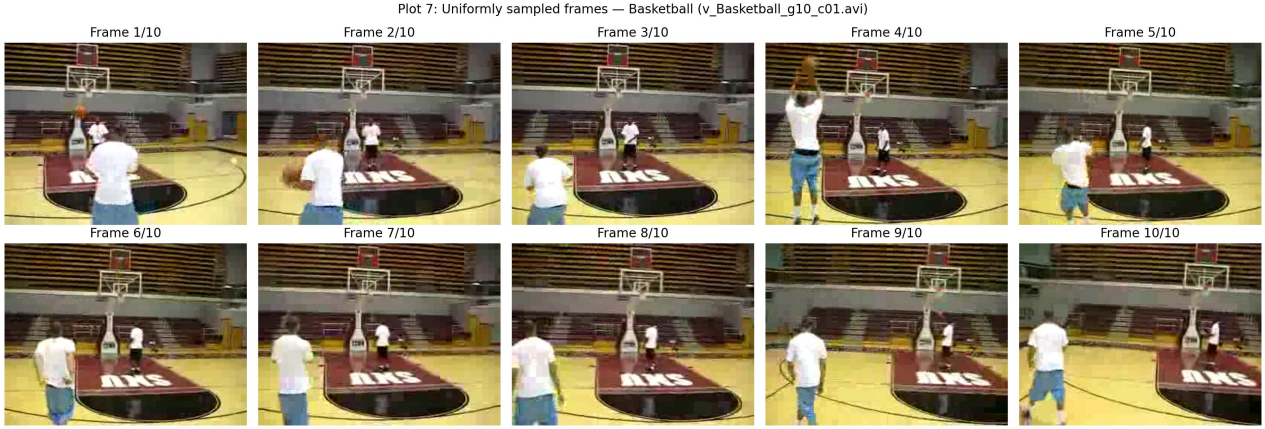


Figure 10: Plot 7: Ten uniformly sampled frames from one training video.

What each stage contributes. The CNN answers “what is in this frame” – a court, a bicycle, a racket, a dog, a human pose – and compresses $224 \times 224 \times 3 = 150,528$ numbers into 1280. The recurrent layer answers “how does this change” – whether the pose oscillates vertically (JumpingJack), translates smoothly (Biking) or shows a single ballistic swing (TennisSwing). Neither stage alone is sufficient: a single frame cannot distinguish a jumping jack from a person standing with arms raised, and raw pixels are far too high-dimensional for a recurrent model trained on 160 examples.

Metric	CNN-LSTM	CNN-GRU
Accuracy (%)	88.00	92.00
Macro Precision (%)	88.35	91.96
Macro Recall (%)	88.00	92.00
Macro F1 (%)	87.54	91.89
Trainable parameters	168,229	126,309
Training time (s)	6.33	6.25

Parameter counts are dominated by the input projection, since $D = 1280$: the LSTM needs $4 \times 32 \times (1280 + 32 + 1) = 168,064$ weights in the recurrent layer alone.

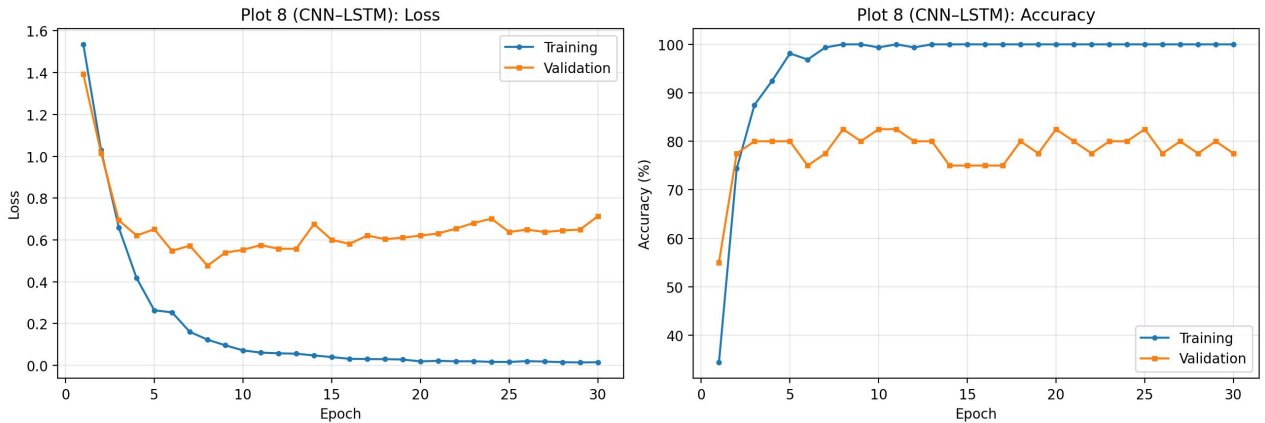


Figure 11: Plot 8: CNN-LSTM training and validation curves.

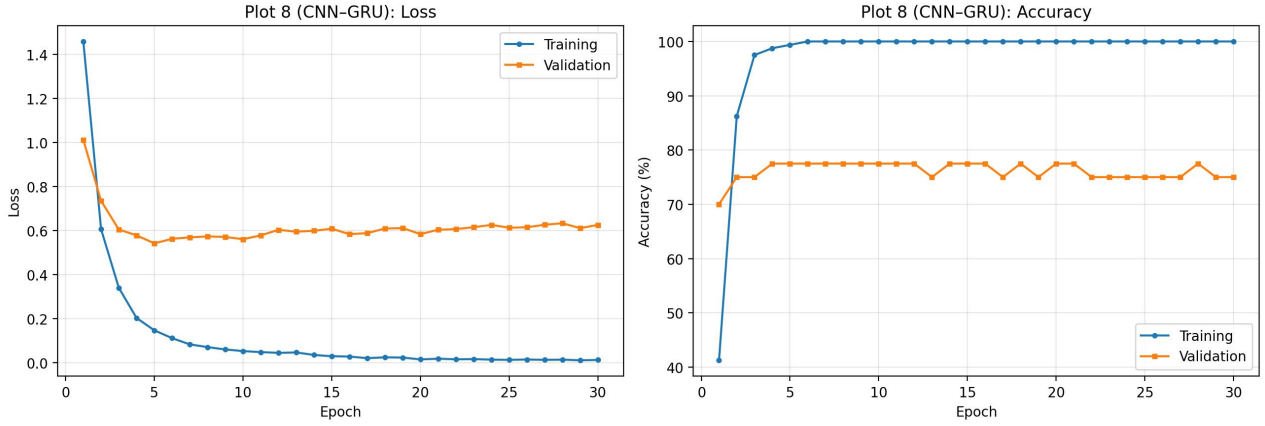


Figure 12: Plot 8: CNN-GRU training and validation curves.

Inference – video convergence. Both heads reach 100% training accuracy by about epoch 8 while validation accuracy settles at 75–82%, and validation loss stops improving after roughly epoch 8 while training loss keeps falling towards 0.01. This is textbook **overfitting**, and it is expected with 160 training videos and 126,000–168,000 trainable parameters. The useful consequence is that most of the learning happens in the first ten epochs; early stopping around epoch 8–10 would give the same test score at a third of the cost.

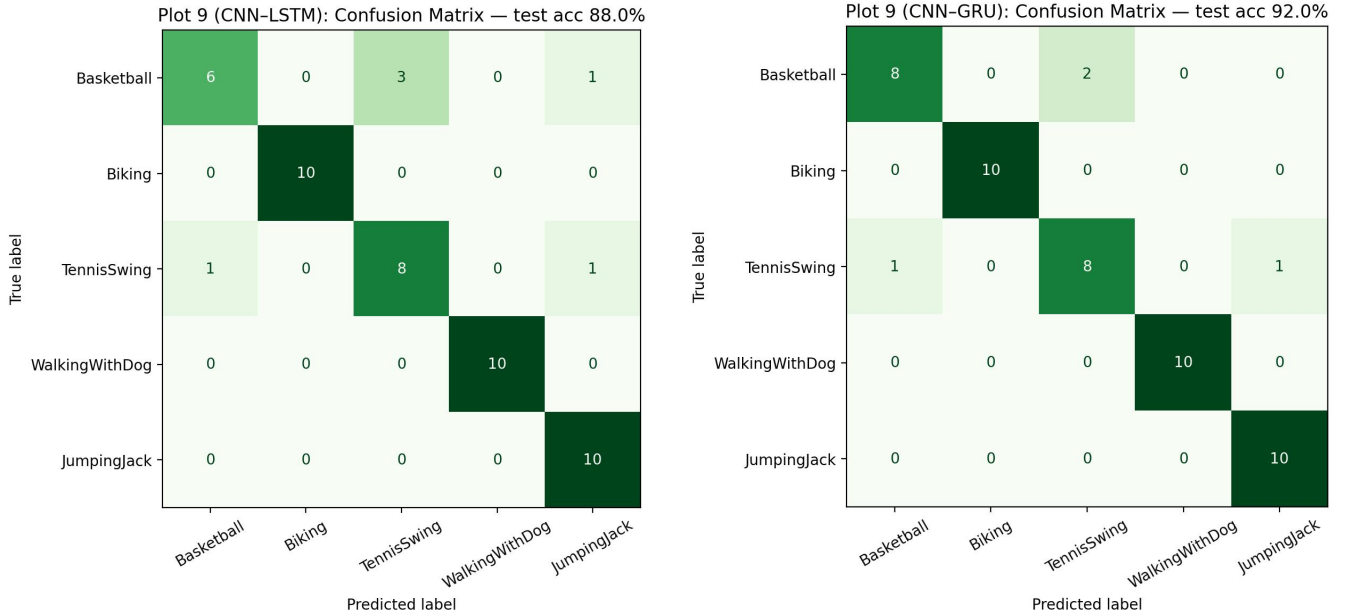


Figure 13: Plot 9: Video confusion matrices for CNN-LSTM (left) and CNN-GRU (right).

Inference – video confusion. With 10 test clips per class, each error costs 10 percentage points of that class’s recall, so the matrices must be read coarsely. The CNN-GRU makes 4 errors out of 50 and the CNN-LSTM 6. Errors concentrate between classes that share scene context and body pose rather than between visually unrelated actions, which is the signature of a pipeline whose discriminative power comes mostly from the frozen spatial features. Classes with a unique object in frame (a bicycle, a dog) are the easiest.

22. Sequence-to-Sequence Learning

A synthetic reversal dataset was constructed: the target is the input read backwards, e.g. $[1, 4, 7, 2] \rightarrow [2, 7, 4, 1]$.

Sequence length	8 tokens (input and output)
Token range	integers 1–30, with token 0 reserved as the decoder START symbol
Vocabulary size	31
Dataset size	12,000 sequences, split 8400 / 1800 / 1800
Embedding dimension	32
Encoder / decoder units	64 LSTM units each
Training	Adam 10^{-3} , batch 64, 25 epochs, teacher forcing
Trainable parameters	53,663

A length of 8 over 30 symbols was used instead of the 4-token illustration in the handout: with 4 tokens the task is solved almost perfectly and the distinction between token accuracy and sequence accuracy becomes invisible, which would defeat the required inference in Section 24.



The encoder compresses the whole input into its final state (h, c) ; that context initialises the decoder, which then generates the output one token at a time. During training the decoder is fed the *true* previous token (teacher forcing); at inference it is fed its own previous prediction (greedy decoding).

Why reversal is a good diagnostic task. The first output token depends on the *last* input token and the last output token depends on the *first*. The model must therefore hold the earliest symbol across the entire encoder and decoder – 16 recurrent steps – which directly tests long-range retention.

23. Sequence-to-Sequence Input and Output

Five test examples produced by greedy decoding:

Sample	Input Sequence	Expected Output	Predicted Output	Correct
1	[18, 2, 21, 1, 13, 23, 2, 4]	[4, 2, 23, 13, 1, 21, 2, 18]	[4, 2, 23, 13, 1, 21, 18, 2]	No
2	[4, 17, 11, 6, 4, 13, 20, 15]	[15, 20, 13, 4, 6, 11, 17, 4]	[15, 20, 13, 4, 6, 11, 17, 4]	Yes
3	[16, 30, 4, 2, 23, 29, 3, 28]	[28, 3, 29, 23, 2, 4, 30, 16]	[28, 3, 29, 23, 2, 4, 30, 16]	Yes
4	[10, 27, 6, 25, 25, 11, 14, 17]	[17, 14, 11, 25, 25, 6, 27, 10]	[17, 14, 11, 25, 25, 6, 27, 10]	Yes
5	[28, 26, 7, 10, 10, 4, 12, 6]	[6, 12, 4, 10, 10, 7, 26, 28]	[6, 12, 4, 10, 10, 7, 26, 28]	Yes

Sample 1 is the instructive failure: the first six tokens are perfect and the last two are swapped ($\dots 21, 2, 18$ instead of $\dots 21, 18, 2$). The model knows *which* symbols belong at the end but has lost their order – precisely the oldest information in the encoder.

24. Sequence-to-Sequence Evaluation

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}, \quad \text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Measure	Value
Token accuracy (greedy decoding)	93.90%
Sequence accuracy (greedy decoding)	81.00%
Token accuracy with teacher forcing	97.42%
Final training loss	0.0821
Final validation loss	0.1211
Final training token accuracy (teacher forced)	98.63%
Final validation token accuracy (teacher forced)	96.99%
Training time	30.9 s

Per-position token accuracy under greedy decoding:

Output position	1	2	3	4	5	6	7	8
Accuracy (%)	99.44	98.83	98.28	97.22	95.50	91.44	86.22	84.28

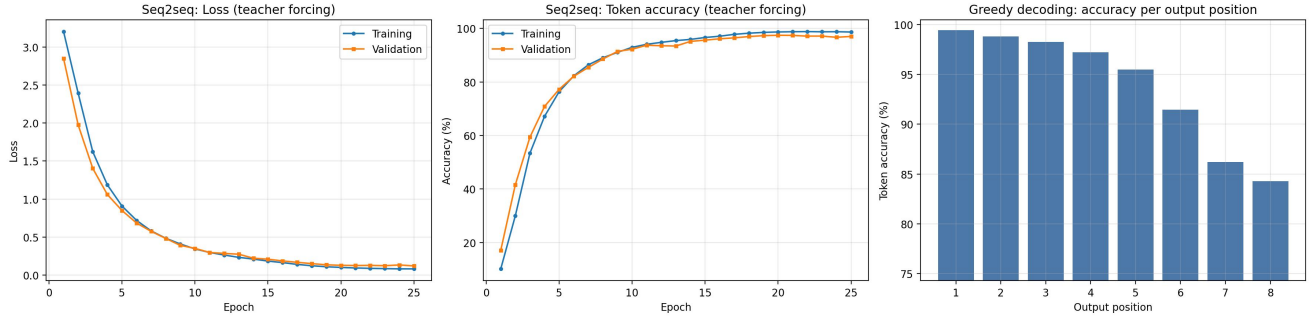


Figure 14: Seq2seq loss, teacher-forced token accuracy, and per-position accuracy of the greedy decoder.

Required inference: why sequence accuracy is much lower than token accuracy.

- A sequence is correct only if *all eight* tokens are correct, so errors combine by an “and”, not an average. If the eight token errors were independent with per-token accuracy 0.939, the expected sequence accuracy would be $0.939^8 = 60.45\%$.
- The observed sequence accuracy is 81.00%, well *above* that independence estimate. Errors are therefore clustered: a small subset of sequences is wrong in several positions at once (like Sample 1, where two positions fail together), while the majority are entirely clean. Token accuracy alone cannot reveal this structure.
- The per-position table shows where the errors live: accuracy falls monotonically from 99.44% at position 1 to 84.28% at position 8. Output position 8 must reproduce input token 1, which has travelled through the whole encoder and the whole decoder, so it is the longest-range dependency in the task and the first to break.
- Greedy token accuracy (93.90%) is also lower than teacher-forced token accuracy (97.42%). At inference the decoder consumes its own outputs, so one wrong token corrupts the context for every later token – exposure bias. This is a second, distinct mechanism that pushes sequence accuracy down.
- **Practical conclusion.** For any task where the whole output must be right (translation, code generation, command parsing), sequence accuracy is the metric that matches the user’s experience, and it degrades roughly exponentially in output length. Attention, which lets the decoder look back at every encoder state instead of relying on one fixed context vector, is the standard fix for the position-8 degradation seen here.

25. Consolidated Results

Classification models (HAR test set: 462 windows, 6 classes; video test set: 50 clips, 5 classes):

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters	Time (s)
RNN	83.55	82.87	82.48	82.50	1,974	27.93
LSTM	94.81	95.37	95.03	94.92	6,006	22.26
GRU	98.48	98.65	98.50	98.53	4,758	21.69
CNN-LSTM	88.00	88.35	88.00	87.54	168,229	6.33
CNN-GRU	92.00	91.96	92.00	91.89	126,309	6.25

Precision, recall and F1 are macro-averaged. The video rows are not directly comparable with the HAR rows: different dataset, 5 classes instead of 6, 50 test samples instead of 462, and a frozen 2.2 M-parameter CNN in front that is not counted in the parameter column.

Sequence-to-sequence task:

Model	Token Acc. (%)	Sequence Acc. (%)	Train Loss	Val Loss	Parameters	Time (s)
Encoder-Decoder LSTM	93.90	81.00	0.0821	0.1211	53,663	30.93

26. Required Inferences (Summary)

1. **Temporal pattern in the sensor signals.** Dynamic activities produce quasi-periodic oscillations with a gait cycle of 20–30 samples; static activities are flat and are separated only by the gravity offset in `total_acc`. Order carries the discriminative information, which is why a sequence model is appropriate.
2. **Convergence of the RNN.** Slow, still improving at epoch 30, training loss stuck at 0.53 and accuracy at 78.5%. Underfitting caused by vanishing gradients over 128 steps, not by lack of data.
3. **Convergence of the LSTM.** Fast to 90%+ by epoch 11, with an optimisation spike at epochs 19–21 and minimum validation loss at epoch 22. Gates give it access to the full window.

4. **Convergence of the GRU.** Smoothest and most stable of the three; validation loss still at its minimum at epoch 30; plateau at 94.4% validation accuracy from epoch 17.
5. **Overfitting or underfitting.** No overfitting on HAR – all three generalisation gaps are negative (dropout active during training plus a slightly easier validation subject mix). The RNN underfits. The *video* models do overfit clearly: 100% training accuracy versus 75–82% validation from epoch 8, with only 160 training clips.
6. **Activity-wise errors.** LAYING is perfect everywhere. The RNN’s errors are between the three walking classes (35 of 76 errors in the WALKING $\leftrightarrow$ WALKING_DOWNSTAIRS pair); the gated models’ errors are almost entirely SITTING $\leftrightarrow$ STANDING. Dynamic-class confusion is a memory problem that gates fix; static-class confusion is a sensor problem that they cannot.
7. **Effect of sequence length.** $T = 32$ (0.64s) is too short for all models; the big gain is $32 \rightarrow 64$; $64 \rightarrow 128$ is a plateau (GRU +0.2, RNN –1.4, LSTM –1.2 points) while cost rises 16–37%. About two gait cycles is enough.
8. **Parameter and computational differences.** RNN 1,974 / LSTM 6,006 / GRU 4,758 parameters, but training time runs 27.9 / 22.3 / 21.7s – the fewest-parameter model is the slowest, because cuDNN fuses LSTM and GRU and not SimpleRNN. Parameter count and wall-clock are independent axes on a GPU.
9. **Role of the CNN in video understanding.** It reduces each $224 \times 224 \times 3$ frame to a 1280-dimensional semantic descriptor, a $118\times$ compression, using ImageNet knowledge that 160 training videos could never supply. Frozen, it makes the whole experiment run in about a minute and keeps the LSTM/GRU comparison controlled.
10. **Role of the recurrent model.** It converts 10 independent frame descriptors into one judgement about motion – direction, periodicity, ordering. Shuffling the 10 frames would leave the CNN features identical but destroy exactly what the recurrent layer reads.
11. **Token versus sequence accuracy.** 93.90% token accuracy yields only 81.00% sequence accuracy, because a sequence is correct only if all eight tokens are. The independence estimate $0.939^8 = 60.45\%$ is lower than what was observed, proving errors are clustered rather than uniform. Accuracy also decays with output position (99.44% $\rightarrow$ 84.28%), since later outputs correspond to earlier inputs.

27. Discussion Questions

1. **What is a recurrent neural network?** A network with a feedback loop: it processes a sequence one element at a time and maintains a hidden state that is fed back as an input to the next step, $h_t = f(W_x x_t + W_h h_{t-1} + b)$. The same weights are reused at every step, so a single small parameter set handles sequences of any length.
2. **What information is represented by the hidden state?** A learned, fixed-size summary of everything seen so far – a lossy compression of the prefix $x_1 \dots x_t$ containing whatever is useful for the task. In this experiment the 32-dimensional state of the trained GRU encodes properties like “an oscillation of this period and amplitude is in progress” and “gravity is along this axis”.
3. **Why is sequence order important?** Because meaning lives in the ordering. Reversing or shuffling a HAR window leaves every per-channel mean and variance unchanged but destroys the gait cycle that separates walking from walking-downstairs. In the seq2seq task the input and its reversal share the same multiset of tokens and require completely different outputs.
4. **Difference between a feed-forward network and an RNN.** A feed-forward network maps a fixed-size input to an output with no state; it would need the 128 steps flattened into a 1152-vector, has no weight sharing across time, cannot handle variable-length input and has no notion of adjacency. An RNN carries state across steps, shares weights across time, and accepts arbitrary lengths. The trade-off is that the RNN’s computation is sequential and cannot be parallelised over T .
5. **Explain Backpropagation Through Time.** Unroll the network into T copies (one per step) that share weights, run standard backpropagation on that deep feed-forward graph, and sum the gradient contributions of the shared weights over all steps: $\partial L / \partial W_h = \sum_t \partial L / \partial h_t \cdot \partial h_t / \partial W_h$. Gradients for a step $t - k$ must be routed through k intermediate Jacobians.
6. **Why can gradients vanish during BPTT?** The factor $\partial h_t / \partial h_{t-k} = \prod \text{diag}(1 - h_j^2) W_h^\top$ is a product of k terms. Since $\tanh' \leq 1$ and typically $\|W_h\|$ is moderate, each term has magnitude below 1 and the product decays like λ^k with $\lambda < 1$. In the worked example $\partial h_3 / \partial h_1 = 0.254$ already; the same rate over 128 steps gives $\approx 10^{-38}$. Early steps then receive no usable learning signal.
7. **Why can gradients explode during BPTT?** The same product with $\lambda > 1$ (large recurrent weights, unsaturated activations) grows geometrically. One batch can produce an enormous update that throws the weights far outside the good region and yields NaN losses. Gradient-norm clipping is the standard mitigation, and unlike vanishing it is easy to detect and cheap to fix.
8. **What are long-term dependencies?** Relationships where the output at time t depends on an input many steps earlier. Examples here: output position 8 of the reversal task depends on input token 1 (16 recurrent steps away), and classifying LAYING depends on an offset present throughout the window. They are hard because the gradient path length equals the dependency length.
9. **What is the role of the LSTM cell state?** It is a protected memory highway. Its update is additive, $C_t = f_t C_{t-1} + i_t \tilde{C}_t$, so $\partial C_t / \partial C_{t-1} = f_t$; with $f_t \approx 1$ information and gradient pass through many steps almost

unchanged. It is kept separate from h_t so that memory can be retained without being exposed as output at every step.

10. **Forget, input and output gates in LSTM.** Forget $f_t = \sigma(\cdot)$ scales how much of C_{t-1} survives. Input $i_t = \sigma(\cdot)$ scales how much of the candidate $\tilde{C}_t$ is written. Output $o_t = \sigma(\cdot)$ scales how much of $\tanh(C_t)$ becomes h_t . All three are sigmoids in $(0, 1)$, computed from $[h_{t-1}, x_t]$, applied element-wise, so each unit gets its own learned, input-dependent retention policy.
11. **Reset and update gates in GRU.** Update z_t interpolates between keeping the old state and taking the new candidate: $h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$; it plays the role of forget and input gates combined. Reset r_t controls how much of the previous state enters the candidate, $\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$; $r_t \approx 0$ lets the unit ignore history and start fresh, which is useful at boundaries between phases of motion.
12. **Compare LSTM and GRU.** LSTM: two states (h, C) , three gates, four weight matrices, more capacity and finer control (keep and write are decided independently). GRU: one state, two gates, three matrices, roughly 25% fewer parameters at the same unit count, usually faster to fit on small data. Empirically in this experiment GRU won on both tasks – 98.53% versus 94.92% macro F1 on HAR and 91.89% versus 87.54% on video – with fewer parameters. On large corpora and very long sequences the two are generally close, with LSTM sometimes ahead.
13. **Why does GRU use a simpler gating mechanism than LSTM?** It was designed to test whether the LSTM’s separate forget and input gates, separate cell state and output gate are all necessary. Tying them $(1 - z$ versus $z)$ preserves the crucial property – an additive, gated state update that protects gradient flow – while cutting a third of the parameters and one matrix multiply per step. Fewer parameters also means less variance, which is exactly why it wins on the small datasets used here.
14. **Why compare RNN, LSTM and GRU under the same settings?** Because otherwise the comparison measures the settings, not the architecture. Identical splits, normalisation statistics, classifier head, optimizer, learning rate, batch size, epoch budget and random seed mean the only varying factor is the recurrent cell, so the 16-point F1 gap can legitimately be attributed to gating. A warm-up run was also added so that one-time kernel compilation was not charged to the first model timed.
15. **What does the confusion matrix reveal that accuracy does not?** The structure of the errors. The LSTM’s 94.81% accuracy hides that STANDING recall is 79.3% while four other classes exceed 95%. It also shows *which* classes are exchanged – walking-variant confusion (a temporal modelling failure) versus SITTING/STANDING confusion (a sensor ambiguity) – which points at different fixes: better architecture in one case, better features or sensors in the other.
16. **Why report macro F1 for multi-class classification?** Macro averaging computes the metric per class and averages with equal weight, so a small or difficult class cannot be hidden behind a large easy one. F1 itself balances precision and recall, which matters when a model over-predicts one label: the LSTM’s tendency to answer SITTING inflates SITTING’s recall and depresses its precision, and F1 reflects both. With the mild imbalance here (1.3:1) macro F1 tracks accuracy closely, but it would diverge sharply under strong imbalance.
17. **What is the significance of sequence length?** It sets how much temporal context the model can use and how long the BPTT path is. Too short and the discriminative pattern is not contained in the window ($T = 32$ cost 5–6 F1 points here); too long and computation grows while optimisation gets harder with no accuracy gain ($T = 128$ was no better than $T = 64$ for RNN and LSTM). It should be chosen from the physical timescale of the phenomenon – roughly two gait cycles for HAR.
18. **Why can a CNN be used as a feature extractor for video?** Because a video frame is an image, and a CNN pretrained on ImageNet has already learned a generic visual hierarchy – edges, textures, parts, objects – that transfers to unseen datasets. Removing the classification head and global-average-pooling the last feature map yields a compact descriptor of frame content. With 160 training videos, reusing those features is the only viable option; training a CNN from scratch would overfit instantly.
19. **What type of information is extracted by the CNN?** Spatial and appearance information within a single frame: objects present (ball, bicycle, racket, dog), scene and background, body pose, colours and textures. It has no notion of what happened before or after that frame.
20. **What type of information is learned by the recurrent network?** Temporal information across frames: how the spatial descriptors evolve – direction and speed of motion, periodicity, the ordering of phases. This is what separates a jumping jack (arms up-down-up) from a static raised-arm pose, or pedalling from walking.
21. **Why are CNN features arranged as a sequence before LSTM/GRU?** Because the recurrent layer’s entire purpose is to model dependence along an ordered axis; it expects (B, T, D) and updates its state once per time step. Feeding the 10 frame vectors as one flat 12,800-dimensional vector would discard ordering, break weight sharing across frames and force a fixed frame count. The sequence layout also generalises to a different number of sampled frames without changing the model.
22. **Explain the encoder and decoder in seq2seq learning.** The encoder reads the whole input sequence and compresses it into a fixed-size context – here the final (h, c) of a 64-unit LSTM. The decoder is a second LSTM initialised with that context; it emits one token per step, feeding the previous token back in, until the output is complete. The split allows input and output lengths, and even vocabularies, to differ. Its weakness is the fixed-size bottleneck, which motivates attention.
23. **What is teacher forcing?** During training the decoder is fed the *ground-truth* previous token rather than

its own prediction, so every step learns from a correct context and the model converges far faster. The cost is exposure bias: at inference the decoder must consume its own outputs, a distribution it never saw in training, so one mistake propagates. This experiment measures the gap directly – 97.42% token accuracy teacher-forced versus 93.90% under greedy decoding.

24. **Differentiate token accuracy and sequence accuracy.** Token accuracy is the fraction of individual positions predicted correctly (93.90% here) – a per-element average. Sequence accuracy is the fraction of outputs that are correct in *every* position (81.00%) – an all-or-nothing measure. Sequence accuracy is always $\leq$ token accuracy and falls roughly exponentially with output length; it is the metric that matches what a user of a translation or parsing system actually experiences.
25. **Why must the test set remain untouched during model selection?** Because every decision informed by the test set – epoch count, unit count, sequence length – leaks information from it, and the resulting score is then an optimistic estimate of performance on genuinely unseen data. In this experiment the validation split (subjects 19, 21, 27, 30) carried all such decisions, and the test split (subjects 7, 8, 11, 15, 20) was evaluated once per final model. The subject-wise split matters for the same reason: with 50% overlapping windows, a random split would have put two halves of the same recording on both sides and inflated every number reported here.

28. Additional Exercises

Exercises 1–6 were executed; exercise 7 is discussed analytically.

Exercises 1–2: effect of the number of recurrent units

Model	Units	Accuracy (%)	Macro F1 (%)	Parameters	Training time (s)
RNN	16	74.24	72.21	790	26.96
RNN	32	83.55	82.50	1,974	27.93
RNN	64	85.06	84.25	5,878	27.57
LSTM	16	92.64	92.71	2,038	21.89
LSTM	32	94.81	94.92	6,006	22.26
LSTM	64	96.75	96.86	20,086	21.99
GRU	16	96.32	96.37	1,670	21.95
GRU	32	98.48	98.53	4,758	21.69
GRU	64	96.54	96.64	15,542	21.81

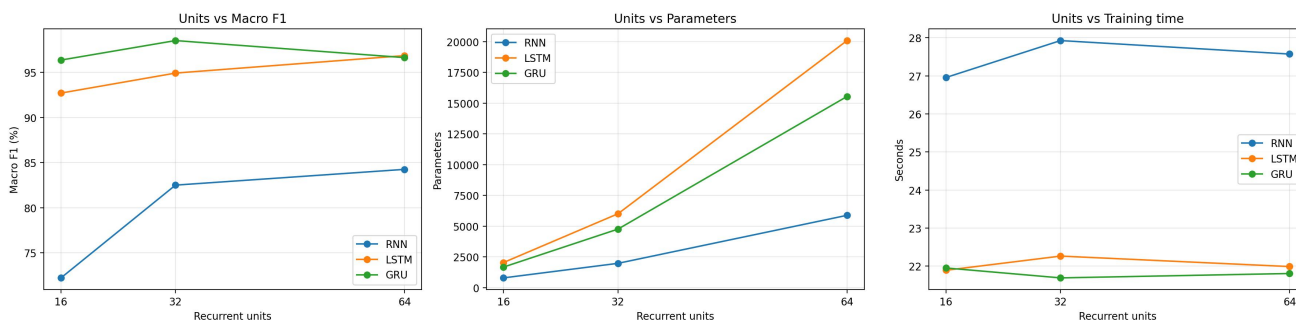


Figure 15: Units versus macro F1, parameter count and training time.

Observations.

- The RNN and LSTM improve monotonically with width; the GRU peaks at 32 units and drops 1.9 points at 64, where 15,542 parameters on 2092 training windows begins to cost more in variance than it gains in capacity.
- Parameters grow roughly quadratically in units (LSTM: 2,038 $\rightarrow$ 6,006 $\rightarrow$ 20,086), while training time is essentially flat (21.9–22.3s). At this scale the GPU is latency-bound on the sequential loop, not compute-bound, so width is nearly free in wall-clock and expensive only in memory and deployment cost.
- **Exercise 2 (GRU vs LSTM at equal units).** GRU beats LSTM at every width – +3.7 points at 16 units, +3.6 at 32 – while using about 25% fewer parameters. At 64 units the ordering flips (96.64 vs 96.86), consistent with the GRU’s advantage being a small-data regularisation effect rather than raw superiority.

Exercise 3: a second recurrent layer

Model	Configuration	Accuracy (%)	Macro F1 (%)	Parameters	Time (s)
RNN	2×32	87.23	86.60	4,054	44.83
RNN	1×32	83.55	82.50	1,974	27.93
LSTM	2×32	96.97	97.07	14,326	31.83
LSTM	1×32	94.81	94.92	6,006	22.26
GRU	2×32	97.40	97.51	11,094	31.08
GRU	1×32	98.48	98.53	4,758	21.69

Stacking helps the two weaker models (RNN +4.1, LSTM +2.2 points) and *hurts* the strongest one (GRU -1.0). Depth adds abstraction capacity that the RNN and LSTM can still use, but the single-layer GRU is already near the ceiling for this dataset, so the extra 6,336 parameters only add variance. The cost is real: training time rises by 43–60%, because the second recurrent layer is another sequential 128-step loop that cannot overlap with the first.

Exercise 4: bidirectional versus unidirectional LSTM

LSTM variant	Accuracy (%)	Macro F1 (%)	Parameters	Time (s)
Unidirectional (32)	94.81	94.92	6,006	22.26
Bidirectional (32+32)	94.81	94.87	11,894	31.79

Bidirectionality gives no benefit here: identical accuracy, 0.05 points lower F1, double the recurrent parameters and 43% more training time. The reason is that the HAR signal is statistically stationary within a window – a walking cycle read backwards is still a walking cycle – so the reverse pass sees essentially the same evidence as the forward pass. Bidirectional models pay off when later context disambiguates earlier content, as in speech or text, not in a repeating periodic signal. This is also a reminder that offline-only processing (which bidirectionality requires) should not be given up for free.

Exercise 5: sequence length versus computational cost

Covered quantitatively in Section 17. Going from $T = 32$ to $T = 128$ raised training time by 52% (RNN, 18.3 → 27.9s), 30% (LSTM) and 25% (GRU), while macro F1 changed by +5.3, +1.5 and +3.4 points respectively – almost all of which was already obtained at $T = 64$.

Exercise 6: LSTM versus GRU on identical CNN features

Covered in Section 21. Both heads consumed the same cached (250, 10, 1280) feature tensor, so the comparison is exactly controlled: CNN-GRU reached 92.00% accuracy / 91.89% macro F1 with 126,309 parameters against CNN-LSTM's 88.00% / 87.54% with 168,229 – the same direction as the HAR result, and for the same reason (less capacity to overfit 160 training clips).

Exercise 7: unequal input and output lengths (analysis)

The implemented encoder-decoder already supports this without architectural change, because the encoder and decoder are separate loops joined only by the context state. Only the data and the stopping rule change: the decoder would be trained with an explicit <EOS> token and generation would run until <EOS> is emitted or a maximum length is reached, instead of the fixed 8 steps used here. Sequence accuracy would then also penalise length errors – a prediction of the right tokens with the wrong length counts as a complete failure – so the gap between token and sequence accuracy would widen further.

29. Expected Outcome

The complete sequence-learning pipeline was demonstrated:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

and the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

Overall conclusion. Gating is what makes recurrence work at $T = 128$: the GRU improved macro F1 by 16.0 points over the Vanilla RNN (98.53% vs 82.50%) while using fewer than half of the LSTM’s additional parameters over the RNN. Beyond that, extra capacity stopped paying – the LSTM, wider GRUs and stacked GRUs were all worse than the plain 32-unit GRU. The remaining errors are not architectural: SITTING ↔ STANDING is a limitation of what an accelerometer can see, and the video model’s ceiling is 160 training clips, not the choice of recurrent cell. Every numerical value in this report was produced by the accompanying notebook on the author’s own execution.

30. Source Code

The complete implementation is present in the following repository:

<https://github.com/D-Rockie/Deep-Learning-Lab-2026.git>

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. Mark Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>